

설비 고장을 예측하는 AI 데이터 분석

Pandas 기초

Pandas 입문과 데이터 불러오기

C O N T E N T S

목차

01 Pandas 입문 / DataFrame 구조

02 CSV 불러오기 / read_csv 옵션

01

Pandas 입문 / DataFrame 구조

표 데이터를 코드로 다루는 이유부터 Series·DataFrame 구조까지

이 과정의 실데이터 — 지하철 공기압축기(MetroPT-3)

오늘 Pandas로 다룰 데이터가 어떤 설비에서 나왔는지 먼저 이해

공정 — 지하철 공기압축기(APU)

지하철 전동차의 공기압축기가 브레이크·출입문에 쓰는 압축공기를 만든다. 압력·온도·전류를 1초마다 기록한 대용량 설비 로그다.

핵심 변수와 물리적 의미

측정시각(기록 시각) · 압축/배출/저장 압력(bar) · 오일온도(°C) · 모터전류(A) · 가동상태(가동/정지). 시각·문자·숫자가 한 표에 섞여 있다.

정상과 이상

압축기가 가동하면 모터전류가 오르고 압력이 차오른다. 오일온도가 비정상적으로 오르거나 압력이 안 차오르면 고장 신호로 본다.

왜 Pandas인가

시각·문자·숫자가 섞인 대용량 설비 로그를 표(DataFrame)로 불러와 head·shape·dtypes·describe로 구조와 상태를 빠르게 파악하기 위해.

표 데이터를 코드로 다뤄야 하는 이유

설비 데이터는 손이 아닌 **코드**로 다뤄야 처리 가능

공장 설비 1대 · 센서 측정 주기

1초에 한 번 측정 → 하루 86,400줄
센서 수십 개 → 하루 수백만 줄

엑셀로 스크롤하며 눈으로 보기에는 불가능한 양
코드로 다루면 데이터가 늘어나도 같은 명령 한 줄로 처리

손작업과 코드 처리의 결정적 차이

코드로 데이터를 다루는 세 가지 이점

엑셀의 수십 번 클릭이 코드 한 줄로 — 대량·재사용·정확성의 차이

01 · 대량 처리

규모의 차이

**수백만 줄도
명령 한 줄로**

온도 평균 구하기·100도 초과 시점 추출 같은
작업이 클릭 수십 번 없이 끝

데이터 규모와 무관한 처리 속도

02 · 재 사용

한 번 짠 코드 다시

**새 데이터에
그대로 재실행**

엑셀은 다음 달 데이터가 오면 처음부터, 코드는 다시 돌리기만 하면 끝

반복 작업의 자동화

03 · 정확성

똑같은 작업을 오차 없이

**코드는
정확히 반복**

고장 신호 하나가 큰 사고로 이어지는 설비 분야에서 특히 중요

수천 칸을 다뤄도 실수 없음

엑셀의 한계와 Pandas의 강점

엑셀과 Pandas는 경쟁자가 아니라 역할이 다른 도구

Q1 엑셀은 언제 좋은가

수백 줄 데이터를 눈으로 보며 직접 고치고 색칠할 때

Q2 엑셀의 한계는

한 시트 최대 **104만 행**, 수십만 줄만 돼도 느려짐

Q3 Pandas의 강점은

수백만 줄도 빠르게, 복잡한 작업도 코드 몇 줄로 반복

눈으로 보고 손으로 고치는 건 엑셀, 많은 데이터를 반복 처리하는 건 Pandas

엑셀과 Pandas의 역할 구분

실무에서는 둘을 함께 — 무거운 처리는 Pandas, 보기 좋게는 엑셀

A · 엑셀

적은 데이터를 눈으로 보기

적은 양을 빠르고 섬세하게 다루는 데 적합한
칼·도마 같은 도구

수백 줄·시각 점검·색칠

B · Pandas

대용량 반복 처리

재료가 많고 같은 작업이 반복될수록 진가가
드러나는 주방 기계

수백만 줄·반복 자동화

C · 함께

Pandas 결과를 엑셀로

Pandas로 무거운 처리를 끝내고 엑셀로 보기
좋게 다듬는 조합

실무 표준 조합

그럼 우리가 다룰 설비 데이터는 실제로 어떤 모양으로 쌓일까

설비 데이터가 표로 쌓이는 방식

행은 한 시점의 측정 기록, 열은 하나의 측정 항목

← 인덱스 (행 번호표)

↓ 열 (측정 항목)

index	측정시각	오일온도	모터전류	가동상태
0	06:38:47	51.3	6.04	가동
1	07:28:21	56.8	0.04	정지
2	08:17:54	55.7	0.03	정지
3	09:07:27	NaN	3.81	가동
4	09:57:01	55.3	0.04	정지

행 → 한 시점 기

행은 시간이 지나며 늘어나고, 열은 측정 항목 수만큼 고정 — 다른 설비 데이터도 같은 구조

Series — 한 줄짜리 데이터

Pandas의 두 그릇 중 작은 그릇 — 표의 한 열에 해당

01 · 정의

Series란

같은 종류 값이
일렬로 늘어선 한 열

온도 열 전체처럼 한 가지 의미의 값이 순서대로 모인 것

표의 한 조각 — 한 열

02 · 인덱스

각 값에 붙는 번호표

0, 1, 2 순서로
번호 부여

출석부 학생 번호에 점수가 적힌 것처럼, 번호로 원하는 값을 바로 찾기

빠른 접근의 키

03 · 활용

DataFrame과의 관계

DataFrame의
열 한 개가 Series

따로 외울 개념이 아니라, 우리가 늘 다루는 표의 한 조각

열 추출 결과 = Series

DataFrame — 표 전체를 담는 그릇

여러 Series가 모이면 우리가 아는 엑셀 표 — 그것이 DataFrame

01 · 정의

DataFrame이란

**여러 Series가
옆으로 모인 표**

설비 데이터 7개 열이 옆으로 붙으면 하나의
DataFrame

엑셀에서 보던 그 표

02 · 강점

무엇이 가능한가

**원하는 행·열을
자유롭게 선택**

온도 열만 보기·100도 넘는 행만 골라내기 같은
작업이 간단한 코드로

행·열 단위 조작

03 · 핵심

CSV와의 관계

**CSV를 불러오면
바로 DataFrame**

앞으로 df라고 쓰면 거의 항상 이 DataFrame을
가리킴

분석 중심 객체

행 · 열 · 인덱스 개념 잡기

행은 가로 한 건, 열은 세로 한 항목, 인덱스는 행 번호표

ROW · 행

가로 한 줄 — 특정 시점의 한 건 기록

그 시각의 오일온도·모터전류·압력이 함께 담김

COLUMN · 열

세로 한 칸 — 하나의 측정 항목

오일온도면 오일온도값만, 모터전류이면 모터전류값만 죽 들어감

INDEX · 인덱스

각 행에 붙은 번호표 — 0부터 시작

엑셀의 왼쪽 행 번호와 같은 역할

idx	측정시각	오일온도	모터전류
0	06:38:47	51.3	6.04
	07:28:21	56.8	0.04
	08:17:54	55.7	0.03
	09:07:27	NaN	3.81
	09:57:01	55.3	0.04

노랑 = 행(row) — 한 시점 기록 1건
초록 = 열(column) — 측정 항목 1개
파랑 = 인덱스(index) — 행 번호표

Jupyter에서 DataFrame 출력 읽기

셀에 변수만 적으면 표가 자동 정렬 — 생략 점·NaN 표시도 익히기

01 · 출력 방법

print 없이도 가능

셀에 변수 이름만
df 쓰고 실행

표가 깔끔하게 정렬되어 자동 출력, print를 쓸 필요 없음

Jupyter의 기본 동작

02 · 가운데 생략

행이 많으면

가운데가 ...로
줄어들어 보임

위·아래 5줄만 보이고 가운데는 점 세 개로 표시, 데이터는 그대로

보기 좋게 화면만 축약

03 · 빈 칸 표시

값이 없는 칸

NaN으로
표시되는 결측치

Not a Number — 측정값이 빠진 자리, 뒤에서 자세히 다룸

결측치 전용 표기

Pandas 데이터 분석 4단계 작업 흐름

불러오기 → 구조확인 → 미리보기 → 통계요약, 4단계의 흐름

STEP 1

불러오기

CSV 파일을 read_csv로 읽어
DataFrame으로 변환

문 열고 들어가기



STEP 2

구조 확인

행·열 수, 자료형 같은 데이터
빠대 확인

구조 둘러보기



STEP 3

미리보기

head·tail로 실제 값을 직접 눈
으로 확인

방 들여다보기



STEP 4

통계 요약

describe로 평균·최대·최소를 한
번에 추출

평수 확인

집을 살펴보는 것과 같음 — 베테랑 분석가도 이 순서를 밟음

4단계 작업 흐름 — 불러오기와 구조 확인

1·2단계 — 분석의 관문이자 뼈대를 보는 단계

STEP 1 · 불러오기

분석의 관문

read_csv로
CSV → DataFrame

불러오기가 안 되면 그 다음으로 못 넘어가므로 사실상 모든 분석의 관문

STEP 2 · 구조 확인

뼈대 파악

shape·columns·info
로 뼈대 파악

데이터의 뼈대를 보는 단계 — 집으로 치면 방이 몇 개인지 둘러보는 일

4단계 작업 흐름 — 미리보기와 통계 요약

3·4단계 — 실제 값을 보고 첫 신호를 잡는 단계

STEP 3 · 미리보기

실제 값과 결측 확인

head·tail로
실제 값 들여다보기

통계 숫자로는 알 수 없는 실제 값의 모습을 직접 확인

STEP 4 · 통계 요약

숫자로 첫 신호 감지

describe로
평균·최대·최소

온도 평균 75도인데 최댓값 132도 같은 첫 신호를 여기서 발견

02

csv 불러오기 / read_csv 옵션

csv 이해부터 경로·인코딩·구분자·열 선택까지 — 모든 read_csv 옵션

CSV 파일이란 무엇인가

CSV는 값을 쉼표로 구분해 저장한 단순한 텍스트 파일

01 · 구조

한 줄이 표의 한 행, 줄 안에서 값들이 쉼표로 나뉨

Comma-Separated Values — 쉼표로 구분된 값

02 · 헤더

첫 줄은 보통 열 이름, 그 아래 줄들이 데이터

맨 윗줄 = 헤더, 아래 = 데이터 행

03 · 호환성

엑셀.메모장.파이썬 어디서나 열리는 범용 형식

설비 시스템·공정 DB·공공데이터 거의 다 CSV

12_METRO_COMPRESSOR.CSV

헤더 — 열 이름
측정시각, 압축압력, 오일온도, 가동상태

데이터 행
2020-02-27 06:38:47, 9.3, 51.3, 가동
2020-02-27 07:28:21, 8.55, 56.8, 정지

CSV를 메모장 · 엑셀로 열어보기

같은 파일도 도구에 따라 다르게 보임 — 내용은 똑같음

Q1 메모장으로 열면

쉼표로 이어진 값이 그대로 보임 — CSV의 진짜 속살

Q2 엑셀로 열면

쉼표 기준으로 칸이 자동으로 나뉘어 깔끔한 표로 보임

Q3 Pandas가 하는 일은

엑셀처럼 쉼표로 칸을 나눠 **DataFrame** 표로 변환

실습 1. CSV 불러오기 워밍업

GOAL

작은 설비 데이터를 CSV로 만들고 메모장 · 엑셀에서 비교

실습 과제

설비 3대 측정값

열: 측정시각, 오일온도, 모터전류

STEP 1

메모장에 쉼표로 데이터 작성 후 CSV로 저장

두 케이스 중 흥미 있는 것을 골라 작성 — 코드 없는 워밍업 실습

실습 — csv 만들기 체크포인트

저장한 파일을 엑셀과 메모장으로 각각 열어 비교

01 엑셀로 열기 — 쉼표가 칸으로 나뉘어 표가 되는지 확인

엑셀이 쉼표를 자동으로 칸으로 변환

02 메모장으로 열기 — 쉼표가 그대로 보이는지 확인

같은 파일이 도구에 따라 다르게 보임을 직접 체감

03 쉼표 하나를 일부러 빼고 다시 열어 칸이 어긋나는지 확인

구분자가 왜 중요한지 직접 체감하는 단계

import pandas as pd

Pandas를 **pd**라는 별명으로 불러오는 코드 — 모든 분석의 첫 줄

NOTEBOOK · CELL 1

```
# 모든 Pandas 코드의 첫 줄  
import pandas as pd
```

import = 외부 도구 불러오기, **as pd** = pandas를 pd라는 별명으로 부르겠다는 약속

실행해도 출력은 없음 — 오류 없이 넘어가면 성공

import의 약속, pd

전 세계 분석가가 통일해서 사용하는 별명 — **pd**

01 · 글로벌 관례

전 세계 분석가가 모두 **import pandas as pd**로 통일 사용

다른 별명을 쓰지 않아야 다른 사람 코드도 쉽게 읽힘

02 · 별명의 효과

긴 **pandas** 대신 짧은 **pd**로 모든 기능 호출

`pandas.read_csv` 대신 `pd.read_csv` — 코드가 훨씬 간결

03 · 위치

import 줄은 노트북 맨 위에 한 번만 작성

한 번 불러오면 그 노트북 안에서 계속 사용 가능

pd.read_csv() 기본 사용법

파일 이름을 따옴표로 감싸 넣으면 결과가 **DataFrame**으로 반환

NOTEBOOK · CELL 2

```
# CSV를 DataFrame으로 - 분석의 시작
df = pd.read_csv("data/12_metro_compressor.csv")
df.head()
```

괄호 안에 파일 이름을 따옴표로 감싸 넣으면 Pandas가 읽어 DataFrame으로 돌려줌 — 결과를 **df**에 담음

read_csv 결과는 반드시 변수에 담아야 다음에 사용 가능

read_csv 결과는 반드시 변수에

변수에 담지 않으면 읽은 데이터가 곧바로 사라짐

01 · 안 담으면

read_csv 결과를 변수에 안 담으면 읽은 데이터가 사라짐

pd.read_csv만 실행하면 읽기는 하지만 저장이 안 돼 사라짐

02 · 담는 법

df = pd.read_csv(...)

등호로 df에 담아야 그 데이터를 계속 사용 — 등호는 오른쪽 결과를 왼쪽 이름에 담는 의미

03 · 비유

도서관에서 책을 꺼내 **책상(변수) 위에 올려두는 일**

책을 꺼내기만 하면 사라지지만 책상에 올려두면 계속 사용 가능

변수에 담기 — df 관례

DataFrame의 줄임말 — 가장 흔히 쓰이는 변수 이름

01 · df 관례

가장 흔한 변수 이름

**DataFrame의
줄임말 df**

pd와 마찬가지로 의무는 아니지만 모두 이렇게 사용

글로벌 관용 표기

02 · 여러 데이터

의미를 붙여 구분

**df_comp
df_dig**

여러 파일을 동시에 다루면 헷갈리지 않게 의미를 붙여 구분

역할별 접미사

03 · 재사용

모든 작업의 출발점

**한 번 담으면
df로 모든 작업**

head로 보고, shape로 재고, describe로 뽑고 —
— 모두 df에서 출발

분석의 중심점

파일 경로 이해

read_csv가 안 된다는 분의 90%는 경로 문제

Q1 상대경로란

지금 작업 폴더 기준 위치, 예: data/12_metro_compressor.csv

Q2 절대경로란

파일의 전체 주소를 처음부터, 예: C:/Users/.../data/...

Q3 어느 것을 쓰나

보통 **상대경로** — 짧고 다른 컴퓨터에서도 잘 작동

경로 오류 — FileNotFoundError

파일을 못 찾으면 나는 오류 — 마지막 줄을 먼저 읽기

JUPYTER · ERROR EXAMPLE

```
df = pd.read_csv("12_metro_compressor.csv")  
  
# 실행 결과 ↓  
FileNotFoundError: No such file or directory:  
'12_metro_compressor.csv'
```

문법 오류가 아니라 **그 경로에 파일이 없다는** 의미 — 오류 메시지 마지막 줄에 핵심

오류는 겁이 아니라 안내문

FileNotFoundException 점검 3단계

파일이 안 보이면 위에서부터 차례로 점검

STEP 1

파일 이름 철자

오타 · 대소문자 확인

equipment ↔ equipment



STEP 2

확장자 .csv

빠뜨리지 않았는지 확인

data.csv ↔ data



STEP 3

위치 확인

작업 폴더인지 data 하위 폴더인지

"data/12_metro_compressor.csv"

read_csv 옵션 — encoding

한글이 깨지면 encoding 지정 — 안 되면 cp949 시도

READ_CSV · ENCODING

```
# 한글 안 깨지게 인코딩 지정  
df = pd.read_csv("data.csv", encoding="utf-8-sig")
```

인코딩 = 컴퓨터가 글자를 해석하는 약속 — 저장 방식과 읽기 방식이 다르면 깨짐

utf-8 계열로 먼저 시도, 안 되면 cp949

한글 깨짐 해결 순서

어떤 인코딩인지 미리 알기 어려움 — 하나씩 시도해 맞는 것 찾기

1순위

utf-8 또는 utf-8-sig

대부분 해결 — 실습 파일도 utf-8-sig로 저장

2순위

cp949

윈도우 엑셀에서 저장한 파일은 보통 cp949

실전 방법

깨지면 하나씩 바꿔 보며 맞는 것을 찾기

미리 알기 어려우니, 시도해 맞는 것을 찾는 것이 현실적

read_csv 옵션 — sep · index_col

구분자가 쉼표가 아니면 **sep**으로 지정

READ_CSV · SEP / INDEX_COL

```
# 세미콜론으로 구분된 CSV
df = pd.read_csv("data_semi.csv", sep=";")

# 특정 열을 인덱스로 지정
df = pd.read_csv("data.csv", index_col="측정시각")
```

sep = separator, 값을 나누는 기호 / **index_col** = 특정 열을 행 이름표로

csv가 항상 쉼표만 쓰는 건 아님 — 세미콜론·탭 파일도 흔함

칸이 뭉치면 구분자 문제

값이 한 열에 뭉쳐 나오면 거의 확실히 구분자가 안 맞은 것

01 · 증상

모든 값이 한 열에 뭉쳐 나옴

df.shape가 200행 1열로 보임 — 정상은 200행 7열

02 · 확인

메모장으로 열어 무슨 기호로 나뉘었는지 확인

문제가 생기면 메모장으로 원본 확인 — 앞에서 배운 습관 그대로

03 · 해결

확인한 기호를 `sep`에 지정

세미콜론 파일이면 `sep=";"` — 실습에서 직접 적용 예정

read_csv 옵션 — nrow · usecols

큰 파일은 **nrow**로 일부 행, **usecols**로 필요한 열만

READ_CSV · NROWS / USECOLS

```
# 위에서 10줄만 빠르게  
df = pd.read_csv("12_metro_compressor.csv", nrow=10)  
  
# 필요한 열만 골라 읽기  
df = pd.read_csv("12_metro_compressor.csv", usecols=["측정시각"])
```

nrow = 위에서 몇 줄만 / **usecols** = 필요한 열만 골라 읽기

열이 많은 파일은 통째로 불러오면 화면이 정신없음

큰 데이터를 다루는 습관

수백만 줄도 위 몇 줄 빠르게 보고 → 필요한 열만 본격 분석

01 · 처음 보는 큰 파일

nrows로 구조만 빠르게 확인

수백만 줄짜리 파일도 위 몇 줄만 빠르게 보며 구조를 파악

02 · 본격 분석

usecols로 필요한 열만 가져오기

관심 있는 센서 몇 개만 — 화면도 메모리도 절약

03 · 주의

nrows는 일부일 뿐 — 실제 분석은 전체 데이터로

위쪽 일부만 본 것을 전체로 착각하면 안 됨

데이터 불러오기 점검 루틴

경로 → 인코딩 → 구분자 → 확인, 네 단계의 진단 흐름

STEP 1

경로

파일이 안 보이면 철자·확장자·
위치 점검

FileNotFoundError



STEP 2

인코딩

한글 깨짐 → utf-8 계열 먼저,
안 되면 cp949

encoding 옵션



STEP 3

구분자

값이 한 열에 뭉치면 sep으로
맞춤

sep 옵션



STEP 4

확인

반드시 head로 제대로 읽혔는
지 눈으로 확인

df.head()

병원 진료처럼 정해진 순서로 점검하면 원인이 빠르게 드러남

점검 루틴 — 단계별 대응

증상별 대응 옵션 — head 확인은 생략 금지

01 · 경로

못 찾음

철자 · 확장자 · 위치 순으로 확인

`FileNotFoundException`

02 · 인코딩

한글 깨짐

utf-8-sig 먼저, 안 되면 cp949

`encoding=...`

03 · 구분자

칸 뭉침

메모장 확인 후 sep에 기호 지정

`sep=";"`

04 · 확인

불러온 직후 head — 생략 금지

오류가 없다고 제대로 불러온 것이 아님 — 구분자가 안 맞아도 오류 없이 뭉칠 수 있음

실습 2. 설비 센서 csv 불러오기

GOAL

`read_csv`로 데이터를 불러와 `head`로 확인

실습 과제

`12_metro_compressor.csv`

200행 7열 — 인덱스 3번 행 오일온도가 NaN

STEP 1

`import` 후 `read_csv`로 담고 `head`로 확인

실습 — 설비 센서 불러오기 체크포인트

불러온 결과를 확인하며 아래 항목 점검

01 기본의 온도 열에서 NaN이 보이는 행 찾기

온도 일부가 NaN으로 표시 — 어느 행에 있는지 직접 찾기

02 변수 이름을 df 대신 sensor로 바꿔도 똑같이 되는지 확인

변수 이름은 자유 — df 대신 sensor도 가능

03 변수에 안 담고 pd.read_csv만 실행하면 어떻게 되는지 비교

데이터가 화면에 한 번 나오고 사라짐 — 다음 줄에서 사용 불가

실습 3. 한글·구분자 깨짐 옵션 다루기

GOAL

encoding과 sep으로 깨진 파일을 직접 해결

실습 과제

세미콜론 구분 파일

sep 없이 읽으면 200행 1열, sep=";"이면 200행 7열

STEP 1

문제 확인 후 sep · encoding 옵션으로 해결

실습 — 깨짐 해결 체크포인트

증상 → 원인 → 해결까지 한 줄씩 기록

01 sep 맞추기 전후 df.shape가 어떻게 달라지는지 비교

sep 없이 200행 1열 → sep 맞추면 200행 7열, shape로 차이 분명

02 인코딩을 utf-8과 cp949로 각각 시도해 맞는 것 찾기

하나씩 바꿔 보며 한글이 제대로 나오는 쪽을 찾기

03 증상 → 원인 → 해결 옵션 순서로 메모

한 줄씩 적어두면 다음에 같은 문제를 빠르게 해결

실습 4. 필요한 열만 골라 불러오기

GOAL

usecols와 nrows로 열 많은 데이터에서 필요한 부분만

실습 과제

센서 3개만 골라 불러오기

```
usecols=[...]
```

STEP 1

usecols로 필요한 열만 읽어 shape 변화 확인

열이 7개 — usecols로 4열만 고르면 200행 4열로 축소

실습 — 열 선택 체크포인트

선택 옵션이 결과에 미치는 영향 확인

01 usecols로 4개 열만 고르면 열이 7개에서 4개로 주는지 확인

shape의 열 숫자가 7 → 4로 줄어듦

02 nrows=20으로 행이 200개에서 20개로 주는지 확인

행이 200 → 20으로 줄어듦

03 열이 많은 데이터에서 이런 옵션이 왜 필요한지 정리

수백 개 열을 통째로 불러오면 느리고 복잡 — 그래서 골라 읽기

실습 5. 경로·옵션 오류 고치기

GOAL

오류 메시지를 읽고 스스로 원인을 찾아 고치기

실습 과제 · 경로 오류 3개

경로 · 철자 · 확장자

data/ 누락, 철자, .csv 누락 — 세 종류의 FileNotFoundError

STEP 1

오류 메시지 읽고 점검 루틴으로 고치기

경로 + data/ + 철자 + .csv 바로잡으면 200행 7열로 정상 출력

실습 — 오류 고치기 체크포인트

오류를 진단하고 고친 과정을 정리

01 각 오류 종류(FileNotFoundError 등) 이름을 말할 수 있는가

FileNotFoundError · UnicodeDecodeError · ValueError

02 오류 메시지에서 단서를 어디서 찾았는지 설명

보통 마지막 줄이 핵심

03 고치기 전후 코드를 나란히 적어 무엇이 달라졌는지 기록

무엇이 문제였는지 분명해짐 — 다음에 같은 오류를 빠르게 해결

실습 6. read_csv 옵션 종합 연습

GOAL

경로 · 인코딩 · 구분자 · 열 선택을 한 번에 적용

실습 과제

세미콜론+한글 파일에서 필요한 열만

sep + encoding + usecols → 200행 3열

STEP 1

여러 옵션을 함께 써서 shape 확인

실습 — 종합 연습 체크포인트

여러 옵션을 함께 쓰는 감각 익히기

01 여러 옵션을 쉼표로 구분해 나열하는 방식 익히기

```
pd.read_csv(path, sep=..., encoding=..., usecols=[...])
```

02 최종 shape가 의도한 행·열과 맞는지 확인

실습 과제 → 200행 3열 / 응용 → 50행 6열

03 옵션을 하나씩 빼 보며 각 옵션의 효과 확인

각 옵션이 결과에 어떤 영향을 주는지 분명히 보임